Inteligencja obliczeniowa w analizie danych | Inżynieria i Analiza Danych

Projekt Monte Carlo

Mateusz Bugdol
Nr indeksu: 419719
Grupa ćwiczeniowa: 1

1. Cel projektu

Celem zadania jest praktyczne sprawdzenie, jak działają algorytmy probabilistyczne na przykładzie metody Monte Carlo. Projekt ma pokazać, że wykorzystując czystą losowość i dużą liczbę prób, jesteśmy w stanie z dużą dokładnością wyznaczyć wartości, których nie da się (lub jest bardzo trudno) policzyć analitycznie. Kluczowym aspektem jest zbadanie, jak zwiększanie liczby losowań N wpływa na stabilizację wyniku i zmniejszenie błędu estymacji.

2. Opis zadania

Zadanie polega na wyznaczeniu przybliżonej wartości liczby π poprzez symulację trafień w obszar ćwiartki koła wpisanej w kwadrat o boku 1.

Kroki wykonywania programu:

- Generowanie par losowych współrzędnych (x, y) z rozkładu jednostajnego w zakresie $[0, 1]$.
- Weryfikację, czy dany punkt leży wewnątrz ćwiartki koła przy użyciu przekształconego równania okręgu: $y \leq \sqrt{1 - x^2}$.
- Statystyczne porównanie liczby "trafień" (k) do całkowitej liczby prób (N).
- Przeprowadzenie wielu serii symulacji (dla N od 100 do 100 000), aby zaobserwować proces zbieżności algorytmu do rzeczywistej wartości $\pi \approx 3.1416$.

3. Wyznaczenie wzoru na π

Aby wyznaczyć wartość π, korzystamy z porównania pól powierzchni dwóch figur geometrycznych:

- Kwadrat o boku $r = 1$, którego pole wynosi $P_{kwadrat} = r^2 = 1$.
- Ćwiartka koła o promieniu $r = 1$ wpisana w ten kwadrat, której pole to $P_{wycinek} = \frac{1}{4}\pi r^2 = \frac{\pi}{4}$.

Prawdopodobieństwo P, że losowo wybrany punkt (x, y) z obszaru kwadratu trafi w obszar ćwiartki koła, jest równe stosunkowi ich pól:

$$P = \frac{P_{wycinek}}{P_{kwadrat}} = \frac{\frac{\pi}{4}}{1} = \frac{\pi}{4}$$

W metodzie Monte Carlo prawdopodobieństwo to przybliżamy jako stosunek liczby punktów "trafionych" (k — leżących pod krzywą) do całkowitej liczby wszystkich wylosowanych punktów (N):

$$\frac{k}{N} \approx \frac{\pi}{4}$$

Po przekształceniu powyższej zależności otrzymujemy wzór, który implementujemy w kodzie:

$$\pi \approx 4 \cdot \frac{k}{N}$$

W programie sprawdzamy warunek trafienia dla każdego punktu, korzystając z nierówności wynikającej z równania okręgu:

$$x^2 + y^2 \leq 1$$

4. Implementacja i wizualizacja procesu estymacji

Przechodzimy do generowania losowych współrzędnych i sprawdzania ich położenia względem łuku okręgu. Całość uzupełnia wizualizacja trafień oraz wykres zbieżności wyników do teoretycznej wartości π.

4.1. Załadowanie bibliotek

- NumPy (np): Nasz główny silnik do matematyki i szybkiego generowania tysięcy losowych współrzędnych (x, y).
- Matplotlib (plt): Podstawowe narzędzie do rysowania wykresów, dzięki któremu zobaczymy naszą ćwiartkę okręgu i rozkład punktów.
- Seaborn (sns): Nakładka na Matplotlib, która sprawi, że nasze wykresy pudełkowe i statystyki będą wyglądać profesjonalnie przy mniejszym nakładzie kodu.
- Pandas (pd): Przyda się do uporządkowania wyników z wielu serii losowań w czytelne tabele przed ich ostateczną analizą.

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

4.2. Implementacja funkcji `estimate_pi`

Funkcja za jednym zamachem generuje n losowych punktów w kwadracie i przy pomocy wektoryzacji sprawdza, które z nich spełniają warunek $x^2 + y^2 \leq 1$, co oznacza trafienie w obszar ćwiartki koła. Zwraca ramkę danych zawierającą nie tylko współrzędne i wynik testu dla każdego punktu, ale przede wszystkim historię zmian przybliżenia liczby π, wyliczaną na bieżąco po każdym kolejnym losowaniu.

[2]:
```
def estimate_pi(n):
    x = np.random.uniform(0, 1, n)
    y = np.random.uniform(0, 1, n)
    inside = np.where(x**2 + y**2 <= 1, 1, 0)
    df = pd.DataFrame({'x': x, 'y': y, 'inside': inside})
    df['id'] = range(1, len(df)+1)
    df['pi'] = 4 * df['inside'].cumsum() / df['id']

    return df
```

4.3. Wielokrotne powtórzenie symulacji.

Uruchamiamy funkcję `estimate_pi` pięciokrotnie dla próby $N = 100\,000$, aby sprawdzić powtarzalność wyników i różne „ścieżki” dążenia do celu. Wyniki są zbierane w liście, a podgląd końcówki jednej z ramek danych pozwala upewnić się, że po dużej liczbie losowań estymowana wartość stabilizuje się blisko teoretycznego π.

[11]:
```
pi_df = []

for _ in range(5):
    pi_df.append(estimate_pi(100000))

final_results = []
for i, df in enumerate(pi_df):
    final_pi = df['pi'].iloc[-1]
    final_results.append({'Seria': i + 1, 'Estymowana wartość PI': final_pi})

summary_df = pd.DataFrame(final_results)
summary_df
```

[11]:

	Seria	Estymowana wartość PI
0	1	3.14728
1	2	3.13916
2	3	3.14212
3	4	3.14432
4	5	3.13724

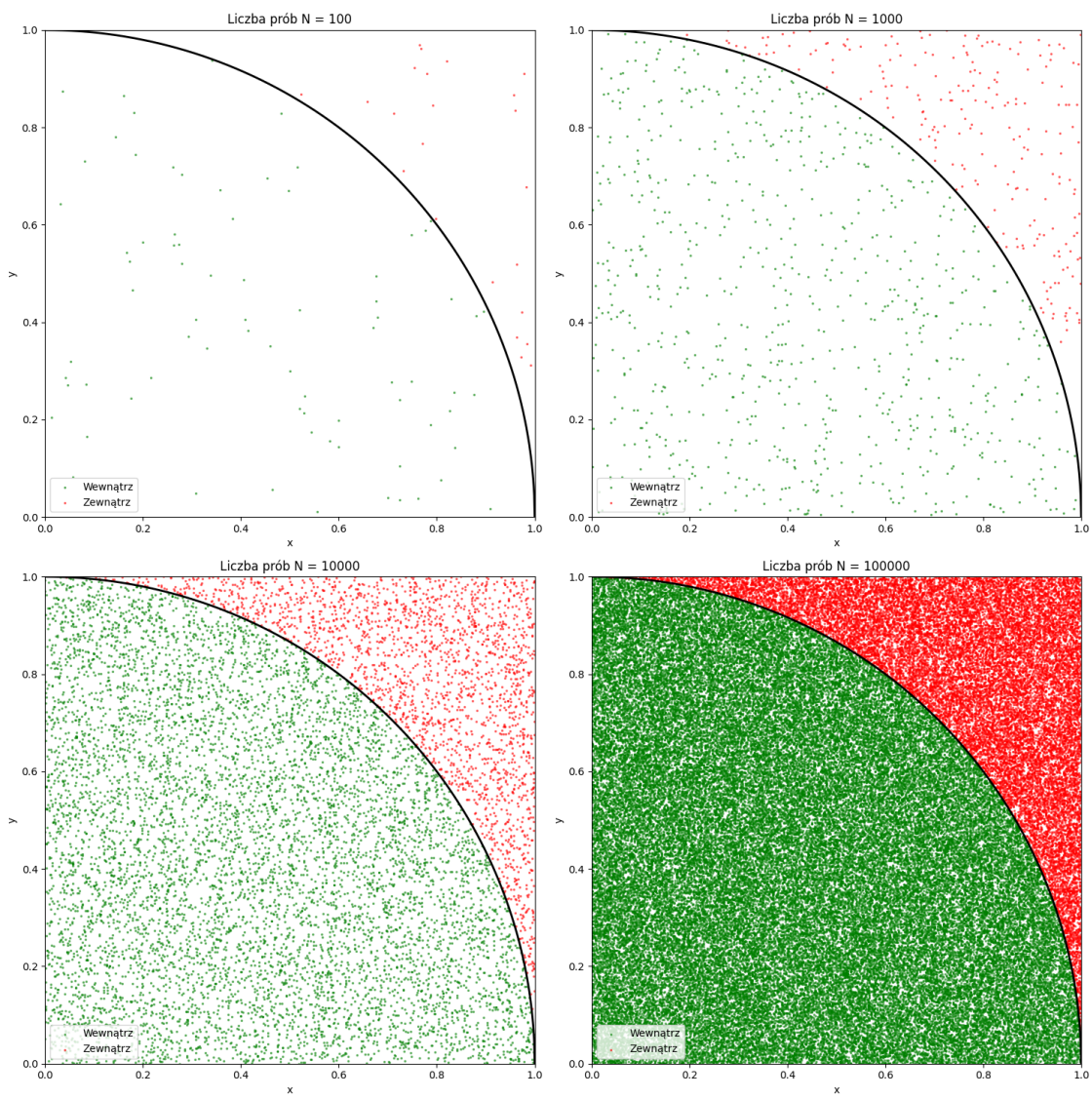
Tabela zawiera końcowe wyniki estymacji liczby π uzyskane w pięciu niezależnych seriach symulacji.

4.4. Wizualizacja rozkładu punktów

Generujemy zestaw czterech wykresów porównawczych, aby zobrazować, jak liczba prób wpływa na wypełnienie badanego obszaru. Kolorami rozróżniamy trafienia wewnątrz ćwiartki koła od punktów leżących poza nią, co pozwala wizualnie ocenić poprawność działania warunku $x^2 + y^2 \leq 1$. Zastosowanie podziału 2x2 pozwala zaobserwować, że wraz ze wzrostem N od 100 do 100 000, kształt ćwiartki koła staje się wyraźniejszy, a statystyczny rozkład punktów coraz lepiej oddaje rzeczywiste pole figury. Dodatkowo czarna linia wyznacza teoretyczną granicę okręgu, potwierdzając, że losowanie odbywa się w zdefiniowanym obszarze badawczym.

[9]:
```
ns = [100, 1000, 10000, 100000]
fig, axes = plt.subplots(2, 2, figsize=(15, 15), facecolor="white")
axes = axes.flatten()

for i, n in enumerate(ns):
    df_temp = estimate_pi(n)
    ax = axes[i]

    df_temp[df_temp["inside"] == 1].plot.scatter(
        x='x', y='y', color="green", s=2, alpha=0.5, ax=ax, label="Wewnątrz"
    )
```

```
    df_temp[df_temp["inside"] == 0].plot.scatter(
        x='x', y='y', color="red", s=2, alpha=0.5, ax=ax, label="Zewnątrz"
    )

    x_arc = np.linspace(0, 1, 1000)
    y_arc = np.sqrt(1 - x_arc**2)
    ax.plot(x_arc, y_arc, color="black", linewidth=2)

    ax.set_title(f"Liczba prób N = {n}")
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    ax.legend(loc='lower left')

plt.tight_layout()
plt.show()
```

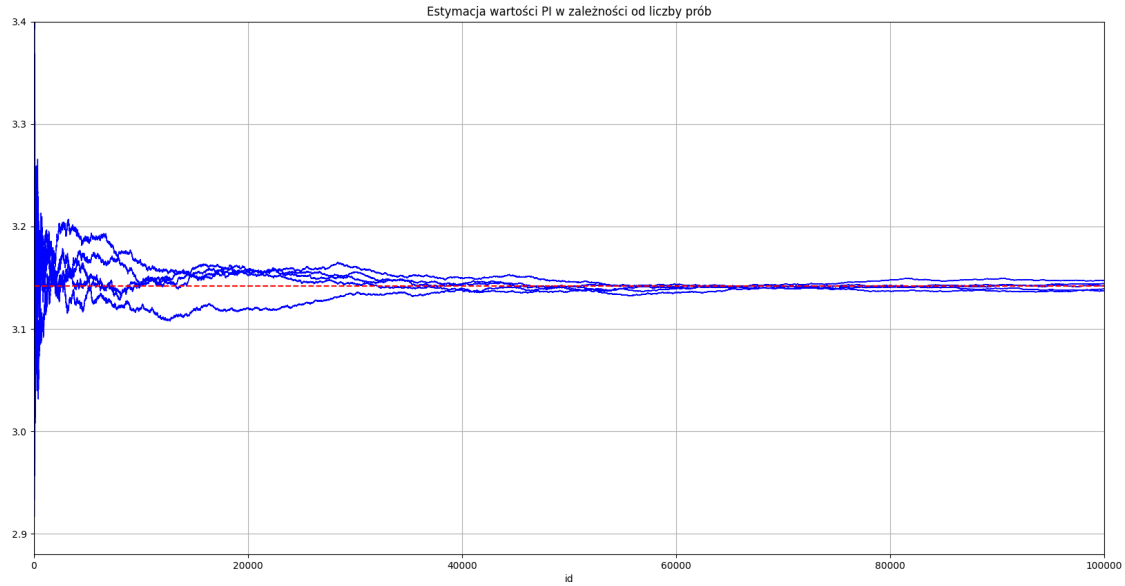


4.5. Wykres zbieżności algorytmu

Generujemy wykres liniowy pokazujący, jak bieżąca estymacja liczby π zmienia się wraz z każdym kolejnym losowaniem dla pięciu różnych serii danych. Czerwona przerywana linia wyznacza teoretyczną wartość π, co pozwala zaobserwować, jak wraz ze wzrostem liczby prób wyniki stabilizują się i dążą do rzeczywistego rozwiązania.

[12]:
```
fig, ax = plt.subplots(figsize = (20,10), facecolor = "white")


for i in range(0, 5):
    pi_df[i].plot(
        x = 'id',
        y = 'pi',
        ax = ax,
        color = 'blue',
        lw = 1,
        legend = False
    )
ax.set_title("Estymacja wartości PI w zależności od liczby prób")
ax.axhline(y = np.pi, color = "red", linestyle = '--')
ax.grid(True)
ax.set_xlim(0, 100000)
ax.set_ylim(2.88, 3.4)
```

[12]: (2.88, 3.4)



Wykres pokazuje, jak wraz ze wzrostem liczby prób wyniki różnych serii symulacji stabilizują się i zbiegają do teoretycznej wartości π (czerwona linia). Początkowe duże wahania zanikają, co potwierdza, że większa liczba losowań bezpośrednio przekłada się na wyższą precyzję estymacji.

5. Podsumowanie

Podsumowując, przeprowadzone ćwiczenie pozwoliło na weryfikację działania algorytmu Monte Carlo jako narzędzia probabilistycznego. Głównym wnioskiem płynącym z symulacji jest potwierdzenie, że wraz ze wzrostem liczby losowań N chwilowa estymowana wartość stabilizuje się i dąży do teoretycznego wyniku, co w naszym przypadku odpowiadało wartości $\pi \approx 3,1416$. Zaobserwowano, że początkowe duże wahania wyników zanikają przy większej skali powtórzeń, co wynika z faktu, że błąd powierzchni pod wykresem jest zależny od odwrotności pierwiastka z liczby prób. Proces ten zobrazowano poprzez wygenerowanie kilku niezależnych „ścieżek" dążenia do wyniku, co pozwoliło ocenić powtarzalność i stabilność przyjętej metody. Wykorzystanie geometrycznego podejścia, opartego na losowaniu punktów w kwadracie i sprawdzaniu ich położenia względem ćwiartki okręgu, udowodniło, że statystyczny stosunek trafień do wszystkich prób pozwala na skuteczne przybliżanie stałych matematycznych. Ostatecznie projekt potwierdził, że metoda Monte Carlo, opracowana przez Stanisława Ulama, jest wydajnym narzędziem w analizie danych, szczególnie tam, gdzie rozwiązania analityczne są trudne do uzyskania. Praca z algorytmem pokazała również korzyści płynące z wektoryzacji obliczeń, która umożliwia błyskawiczne przetwarzanie bardzo dużych zbiorów danych przy zachowaniu wysokiej precyzji estymacji.